

A RESILIENT AND TRUST-BASED OTA FRAMEWORK FOR IOT DEVICES

25-26J-058

Project Proposal Report

Dilmith K.P.H.S

B.Sc. (Hons) in Information Technology Specializing in Cyber Security

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

A RESILIENT AND TRUST-BASED OTA FRAMEWORK FOR IOT DEVICES

25-26J-058

Project Proposal Report

B.Sc. (Hons) in Information Technology Specializing in Cyber Security

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

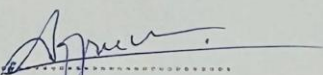
Declaration

Declaration

I declare that this is our work, and this proposal does not incorporate without acknowledgment of any material previously submitted for a degree or diploma in any other university or Institute of higher learning, and to the best of our knowledge and belief, it does not contain any material previously published or written by another person except where the acknowledgment is made in the text. Also, I hereby grant to the Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic, or other mediums. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
Dilmith K.P.H.S	IT21832130	

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.


.....
Mr. Kavinga Abeywardena
Supervisor

11/09/2025
.....
Date

.....
Ms. Dinithi Pandithage
Co-supervisor

.....
Date

Abstract

Updates are crucial for IoT(Internet of Things) devices to keep devices secure, functional and stable. Yet, the typical Over-the-Air (OTA) update mechanisms are known to be vulnerable to reliability problems, which makes the devices exposed to malware or failed installations or even to downgrade attacks that can make them inoperable. In this research project, a Resilient Fail-Safe Update Engine with special attention to resource constrained IoT devices in order to maintain constant availability and secure update of device firmware is introduced. The proposed engine would provide a multi-layer approach that comprises secure validation of firmware, rollback protection and automatic recovery features.

Firmware updates are verified with cryptographic hashes and version numbers prior to being installed on the router, preventing rogue or out-of-date firmware. To ensure resilience, the system is dual-partition designed, allowing an active slot, updated slot, and a golden recovery image. In unexpected events such as, loss of power, crashes, or a failed boot procedure, the engine restores functionality by downgrading to a stable firmware version or reverting to the recovery image to protect the device from bricking.

The Resilient Fail-Safe Update Engine enhance trust in OTA firmware procedures on the one hand, and on the other hand is practical to implement on resource-constrained IoT devices by integrating lightweight cryptographic validation with fault-tolerant methods. The result is a stable, robust, and rollback-immune update system that significantly enhances the device stability and safety of real-world IoT deployments.

Key words: IoT (Internet of Things), OTA (Over the Air), Firmware, Fault-tolerance, Hashing, Rollback

Table of Contents

Declaration.....	3
Abstract.....	4
List of figures.....	6
List of tables.....	6
Introduction.....	7
Background and Literature Survey	7
Background.....	7
Literature Survey	8
Research Gap	10
Research problem.....	14
Objectives	16
Main Objective.....	16
Specific Objectives	16
Methodology	18
Research Methodology and Development Stages.....	18
Overall System.....	19
Individual System	21
Timeline	23
Project Requirements.....	23
Functional Requirements	24
Non-Functional Requirements	25
Testing and Evaluation	26
Commercialization and Entrepreneurship Potential	28
Practical Industry Adoption	29
Budget and Budget Justifications.....	32

References.....	34
-----------------	----

List of figures

Figure 4 . 1 : Research Gap Diagram.....	13
Figure 7 . 1 : Overall System	19
Figure 7 . 2 : Individual System.....	21
Figure 8 . 1 : Timeline Gantt chart.....	23

List of tables

Table 11 . 1 : Budget and Budget Justification.....	33
---	----

Introduction

The technology that we have in today depends on the use of IoT (Internet of Things) to build smart systems such as healthcare networks, smart homes, industrial control systems, and transportation infrastructure.

Most IoT devices perform their functions completely remotely or without any guidance once installed. The security and functionality of these devices rely on their firmware, a low-level software that interacts with the hardware [1], [2]. This is the reason firmware must be safe, stable, and updated.

Over-the-Air (OTA) updates can be used to keep firmware up-to-date. With the help of OTA, security fixes, upgrades, and patches may be remotely delivered. But OTA updates come with some critical risks [3]. If the update becomes corrupted, interrupted, or tampered, the device may become vulnerable to attacks or be damaged, which will cause failures in operations [4]. This failure can cause severe or even life-threatening issues in safety critical systems such as medical devices, smart energy systems or industrial sensors [5], [6]. One solution to this problem is using the fail-safe update engine to correct this. It secures the device to avoid the risk that, in case of an update, the device fails to update and enters a broken state, therefore being able to rollback to a stable version safely. This must be protected against rollback attacks,

To achieve secure rollback, the following are needed: have secure versioning, have recovery logic, and provide validation of integrity prior to the new firmware being operational. In contrast with older approaches that assume ideal conditions, a fail-safe approach anticipates a real-world misbehavior such as loss of power, memory corruption, or network outage during the update [7], [8]. This research aims at developing an IoT-based Resilient Fail-Safe Update Engine. It is proposed to create a rollback-safe design with secure firmware versioning to enhance the long-term reliability, security, and trust of IoT ecosystems.

Background and Literature Survey

BACKGROUND

IoT is powering up and running key services within the industries of healthcare, manufacturing, transportation, and smart cities. Unlike traditional devices, IoT devices are small, they have limited resources, and they operate in areas that cannot be easily accessed by people as compared to traditional devices [9]. Due to this fact, both functionality of the device and the safety of its communication is controlled by firmware. Ensuring these devices are up-to-date and secured by updating their firmware frequently.

Traditional methods of updating firmware are used manually by updating devices using physical cables. This method is impractical with thousands of devices being deployed in the field. Instead, new firmware is remotely broadcast using a wireless network with OTA updates [10], [11]. OTA updates are convenient but generate novel issues. A failed update could be caused by a power failure, a poor connection, or damaged data that results a device worthless. The update process is also susceptible to attack by hackers who can send malicious firmware. It all negatively impacts users of the devices: their downtime, the high cost of replacement, and security failures may occur. Firmware update systems need to be secure against attacks from the outside but capable of recovering from failures. A fail-safe update engine achieves device protection by the following:

- Rollback protection prevents reinstallation of old, vulnerable firmware by the attackers.
- Recovery logic enables devices to roll back to a prior stable version in the event of any update fails to perform effectively.
- Secure versioning ensures only validated and trusted firmware is applied.
- Integrity validation checks the correctness and completeness of the firmware before activation. The firmware is first verified as legitimate, complete, and correct.

Those mechanisms will improve user trust and user experience by reducing device downtime and providing continuous IoT devices functionality even in a unexpected security events.

LITERATURE SURVEY

There have been some studies and industry practices which are aimed at making IoT firmware update systems stronger. Nevertheless, numerous current options targets security and pay less attention to resilience to failures.

1. Conventional OTA Systems

The OTA update mechanism in IoT initially concentrated on remote delivery. They have effectively cut the price of manual interactions, but they do not have recovery measures. **Saad El Jaouhari and Eric Bouvet (May 2022)** further researched to address the issues in the Over-The-Air Updates by reviewing surveys, challenges and discussions but yet when the updating stopped because of the lost power or connection, the device might remain in non-operational state is not addressed [12], [13]. Such limitations increased the need of resilience in addition to convenience.

2. Safe OTA Frameworks

Some research studies suggested on adding cryptographic safeguards in firmware delivery. As an example, it helps to ensure the firmware originated, is authenticated with digital signatures, hash validation, and public key infrastructure (PKI). **Alexander Yohan, Nai-Wei Lo and Liem Peter Santoso (June 2025)** have introduced a robust and efficient blockchain-based framework for updating firmware in IoT devices [14]. These methods enhance the strength of integrity and legitimacy; they do not address the issue of the installation failure of updates and

recovery directly. Even the secured firmware could be unusable if only part of the firmware is installing during an outage.

3. Installing Rollback Protection Mechanisms

Recent solutions, like those provided in trusted computing platforms, aim to prevent attackers from downgrading firmware to older versions to susceptible releases. The rollback protection mechanisms have discussed by **Manuel Huber, Stefan Hristozov, Simon Ott, Vasil Sarafov, and Marcus Peinado (May 2020)** but, this is done by either enforcing monotonic counters or secure version identifiers. Nevertheless, the rollback protection is not enough because there are scenarios that latest firmware fails in the installation process [15].

4. Dual-Bank and A/B Update Systems

More current mobile devices and some IoT products use dual-bank (or A/B) partitioning, in which the firmware is written to an inactive partition when the active one is executing. After the update is confirmed as a legitimate version, the device changes over to a different bank. In case of an update failure the system automatically reverts to the older bank. **Google's Android Things platform** is currently uses similar approach but the limitation is it requires additional flash devices which is making it unsuitable for resource constrained IoT devices [16]. Although efficient, such systems usually need more memory and they might not be viable on resource-limited IoT devices with small flash storage options.

5. Lightweight Recovery Approaches for Constrained Devices

Some researchers (**De Simon et al., May 2025**) propose lightweight fail-safe designs for microcontrollers with restricted resources [17]. These involve checkpointing firmware states or maintaining minimal recovery code in ROM. However, such methods often sacrifice **update speed** or **security validation**, leaving gaps in ensuring both resilience and protection against malicious updates.

6. Industry Implementations

Large-scale IoT platforms, such as those by **Microsoft Azure IoT and AWS IoT**, provide OTA update management systems [18], [19]. While they incorporate device authentication and monitoring, the recovery logic is still largely dependent on the hardware capabilities of the device. This means that smaller and cheaper IoT devices may not benefit fully from these industrial frameworks.

Besides the survey above, a number of the existing OTA frameworks and platforms have been explored to determine their relevance in a secure and resilient IoT environment. Table below summarizes their strengths and weaknesses:

Solution	Strengths	Weaknesses
Mender OTA	Supports delta updates, rollback mechanisms.	Lack of Zero-Trust principles.
Eclipse Hawkbit	Flexible updating mechanism and management.	Limited resiliency
Tuya IoT	Supports large-scale systems, secure cloud-based option.	Limited transparency, not optimized for constrained IoT devices.

Table 3. 1. 2. 1 : Current solutions strengths and weaknesses

Summary of Literature Findings:

- Many existing solutions address **security (authentication, encryption, integrity)** but do not sufficiently cover **resilience (rollback, recovery, failure handling)**.
- Some designs such as A/B systems ensure resilience but are **resource-heavy** and unsuitable for low-cost IoT devices.
- Lightweight recovery approaches exist but often trade off **security validation**.
- Industrial platforms provide strong security but **lack universality across constrained IoT hardware**.

Research Gap

The IoT ecosystem has been widespread in **over-the-air (OTA) firmware update** systems. **Automated firmware distribution, version management, and rollback solutions** (like **Eclipse Hawkbit, Mender**, and commercial cloud-based solutions like **Tuya IoT**) include these solutions. Nevertheless, the solutions have significant **restrictions** when considered in the context of **Zero-Trust security concepts** [20].

The survey of academic and commercial practices seems to suggest that, though IoT firmware updater systems have advanced a fair bit, resilience toward the failure of updates is an important problem that is still critically unexplored, and which is particularly relevant to resource constrained IoT equipment. The gaps that are seen are as follows:

1. Assumption of Trust

A lot of current OTA systems have an initial trust relationship given at the provisioning of the device. Subsequent updates are supposed to be safe once authenticated. This exposes systems to attacks in case the credentials of devices are stolen or in case they are attacked by insiders who have a weak trust boundary.

2. Focus on Security and lacks focus on Reliability

The majority of the current OTA frameworks heavily concentrate on security solutions of encryption, authentication and digital signature [21]. Although they prevent the use of malicious firmware injections, they cannot ensure that devices work when an upgrade fails. Even an update remains incomplete or interrupted, and the failure is due to a power failure, loss of connectivity, or a corrupted transmission, a device may become permanently unusable. This presents a research imbalance which makes the reliability as a sub-ordinary feature compared to security.

3. Recovery Logic Free Rollback Protection

There are also rollback protection mechanisms, in particular monotonic counters, and secure version identifiers, that ensure that there is no way to roll back firmware releases to lower, vulnerable versions. The issue is rollback support is not a guaranteed system resiliency to partial failures, including power loss or flash corruption during installation, resulting in the bricking of the device and if the latest update fails to operate correctly there are no options to restore previous most stable version.

4. Dual-Bank (A/B) Partitioning is resource intensive

More modern solutions, like dual-bank or A/B update systems, offer resilience due to the maintenance of two images of the firmware. But these approaches have the drawback of demanding large flash memory and hardware requirements that are not always readily available in cost-limiting or constrained IoT devices. As an example, most microcontrollers on smart sensors, wearables, and industrial controllers are not capable of supporting the additional storage overhead. Hence, there is a gap between lightweight resilience mechanisms for such constrained devices.

5. High Cost

Commercial OTA solutions are closed-source and expensive, or vendor-specific, making it challenging to use in low-budget or resource-limited IoT deployments such as in **Sri Lanka**.

6. Lightweight Fail-Safe Methods Trade Off Security

Embedded devices that attempt to minimize costs through fail-safe designs may avoid security validation (e.g., integrity checking or cryptographic verification is omitted in order to save memory). This exposes devices to firmware-based malicious attacks, thus defeating the intent of secure OTA updates. No sufficiently well-established model exists to achieve a combination of lightweight fail-safety and strong security guarantees in a balanced manner.

7. Inconsistent Secure Versioning Mechanisms

Although versioning systems do exist in enterprise-level solutions, they are not always consistent and do not have a standardized implementation in an embedded environment with IoT. Several low-cost IoT devices continue to use simple counters or flags unencrypted, and attackers can use this to re-plant an older firmware. They need to perform a lightweight but tamper-resistant cryptographically enforced versioning that emerges in this gap.

8. Solutions of Industry Are Not Applicable

All IoT management platforms that are hosted on the cloud (such as AWS IoT, Azure IoT) have powerful orchestration of the updates, but resilience logic depends on the hardware used. Smaller devices with less flash and RAM (system memory) would not find it useful to use these enterprise frameworks. This has the effect of leaving a deployment gap, in which large and resource-rich devices are relatively well-secured. Still, billions of small IoT devices in domestic smart devices, agriculture, and industrial monitoring are largely unprotected against permanent update failure.

Consequently, although the previous studies have covered the automation of updates and encryption, a severe gap still exists in providing a **rollback-safe, Zero-Trust-compliant OTA update model** that is **lightweight** and **resilient** to **resource-constrained IoT** settings. The Project will eliminate that gap by integrating the **mTLS authentication**, the **ECC/AES-CCM encryption**, **anomaly-detection**, and **rollback-safe-recovery** algorithms into a single architecture.

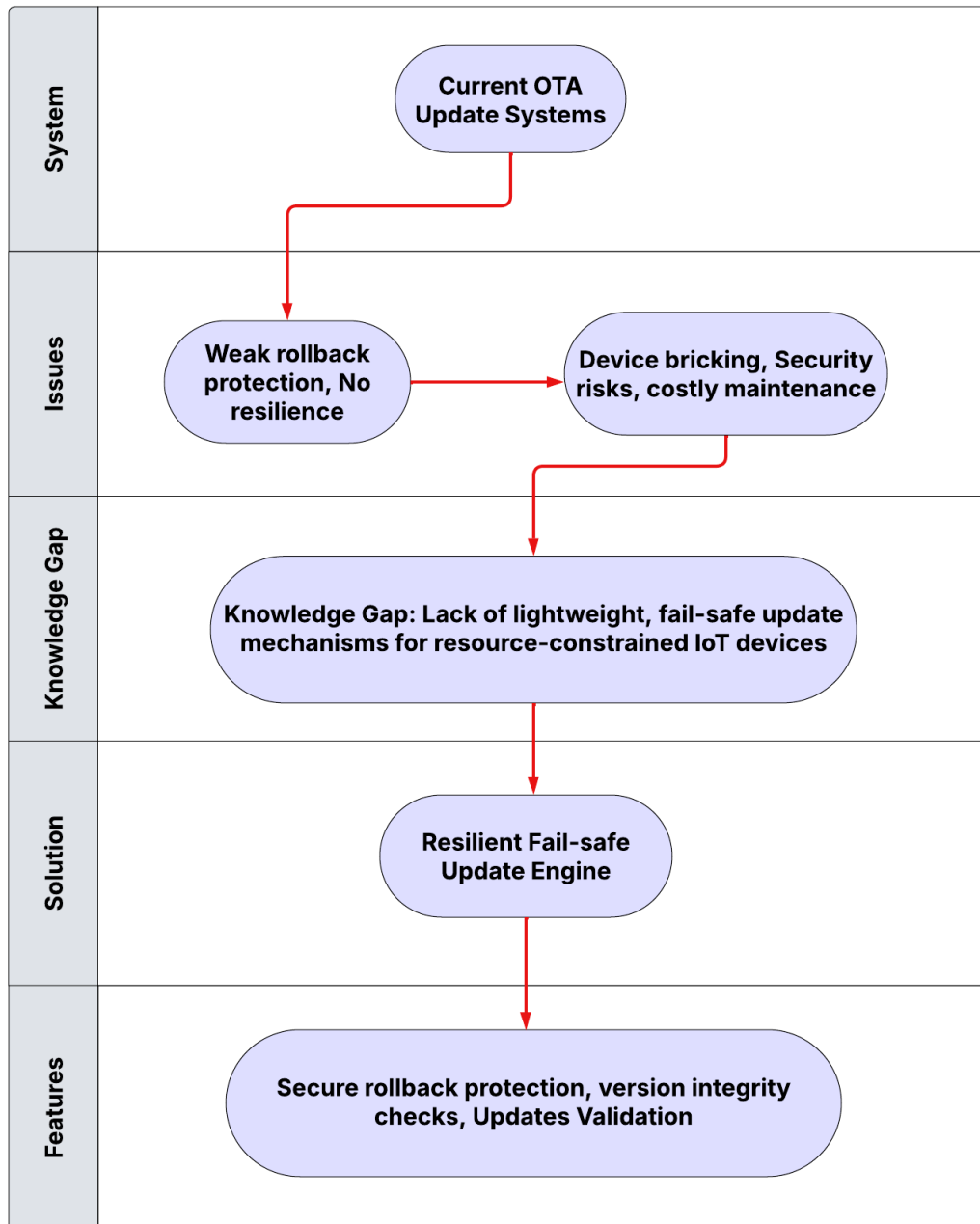


Figure 4 . 1 : Research Gap Diagram

Research problem

The growing use of Internet of Things (IoT) devices in fields that include smart homes, healthcare, transportation, agriculture, and industrial automation has meant that updating the firmware that operates such systems is a vital procedure to ensure the security and continued functionality of a device remains viable. Over-the-Air (OTA) vulnerability and feature update is secured, and does not require physical access. Nevertheless, even though significant advances have been made in the field of secure delivery (e.g., encryption, authentication, and digital signatures), not a lot of attention has been paid to firmware update resilience. In this respect, during the update of the firmware of an IoT device, it is highly susceptible to various breakdowns. The fault may be caused by the interruptions of power supply, network connectivity, corruption of data, or an unexpected system crash that leaves the device in a bricked or useless state. These devices can also become bricked to the point of manual reprogramming or physical repair, an option that would be infeasible in large deployments of IoT devices. Although recovery features using rollback protection and protection mechanisms such as secure versioning have been introduced, these approaches tend to be applied only where the recovery mechanism is readily available, such as during a failed installation (a common use case in resource-constrained devices where two partitions or redundant firmware banks are not possible).

Meanwhile, lightweight recovery schemes tend to be insecure due to the tradeoff of security in cryptographic verification. This poses a severe gap between the security and reliability of the IoT OTA systems. In this way, the research problem can be outlined in the following way:

These are the significant issues in the research problem,

1. Maintaining Resilience in Constrained Environments.

- Conventional dual-bank or redundant update approaches are impossible in many IoT devices with limited memory, storage, and processing capabilities.
- The challenge is how to design a lightweight resilience mechanism operating under such constraints, which is also effective.

2. Security vs. reliability Balance

- Firmware authenticity is guaranteed by strong cryptographic validation (e.g. digital signature), which is a heavier computation.
- The lightweight recovery processes can reduce security, resulting in a trade-off that needs to be resolved to have resilience and security simultaneously.

3. Rollback-Safe Recovery

- Rollback protection stops attacker attempts to reinstall old and vulnerable firmware, but the most recent firmware may fail during the update.
- The challenge is to construct a recovery mechanism that will not reduce rollback protection and keep the devices continuously functional and secure.

4. Secure Firmware Versioning (SFV)

- Often, IoT systems use tracking the software or microcode versions in their software stacks using simple counters or flags, which can be tampered.
- Cryptographically bound versioning techniques are urgently required, but must be lightweight enough to be used on constrained devices.

5. Applicable to any IoT Ecosystems.

- The systems supporting IoT large enterprise (e.g., AWS IoT, Azure IoT) support resilience mechanisms. Still, low-cost devices cannot adopt such mechanisms in areas such as smart agriculture, home automation, and medical monitoring.
- The difficulty lies in developing a universalized fail-safe update engine that can be embedded into an extensive set of devices with little hardware needs.

Objectives

Main Objective

The main objective of the research is to design and implement a Resilient Fail-Safe Firmware Update Engine for resource-limited IoT devices to offer secure, rollback-safe, and recovery-oriented firmware updates, limiting the risk of procedure malfunction during the process of the Over-the-Air (OTA) update processing, while guaranteeing security. This aims to address the much-needed gap in security and reliability seen in IoT update systems by coming up with a lightweight yet robust recovery system that can run within strict limitations in hardware.

Specific Objectives

1. To develop and implement a lightweight rollback protection mechanism

Description: Make it safe to avoid future installation of outdated or vulnerable firmware, even in the cases of recovery processes. This includes the development of cryptographic methods of version verification, which are sufficiently lightweight to be implemented on microcontrollers with flash and RAM limitations.

Rationale: Stopping rollback attacks is the key to ensuring long-term device security. Rollback safety needs to be mitigated carefully against resilience so that devices can recover without being locked into a broken state.

2. To create a secure firmware versioning system

Description: Introduced cryptographically bind firmware versions to device identity to update integrity so that attackers cannot forge firmware version information. The versioning must be tamper-resistant but computationally efficient.

Justification: Conventional version counters/flags are vulnerable to tampering in untrusted environments. Secure versioning system prevents malicious downgrade attempts and ensures integrity.

3. To implement a resilient update and recovery engine

Description: Develop a recovery logic that will allow updating the device, detect failure to update (e.g. due to power failure, corruption or incomplete update), and safely reset the device to previous, known stable firmware.

Justification: Existing solutions usually brick devices when an update fails. A dedicated recovery engine supports continuity of service and reliability; manual intervention is expensive or even impossible in large-scale deployment.

4. To guarantee lightweight cryptographic validation during the update process

Description: Use an effective cryptographic algorithm (e.g., digital signatures, hash) to validate the authenticity and integrity of firmware and ensure low computational overhead.

Rationale: Resiliency and security are essential, and cannot focus only on one aspect. Using a low-resource required cryptographic algorithm helps to achieve an ideal balance of security and resiliency for resource-constrained IoT devices

5. To assess the framework in a realistic failure environment

Description: Try the fail-safe update engine on popular IoT hardware (e.g., ESP32, STM32) in the following situations: power-off, poor connection, damaged packets, and unexpected reboots. Measure recovery length, resources maxed out, and resilience in general.

Rationale: It is not sufficient to provide a theoretical solution; this solution to the proposed engine must be proven under real-world conditions to show its practical applicability and that it can be deployed in a low-cost IoT solution.

6. To ensure the proposed approach is universal and scalable

Description: Design the engine in a modular manner allowing it to be used in various IoT domains such as smart home, agriculture, medical and industrial automation applications, regardless of the type of device.

Rationale: Most IoT resilience solutions are platform-specific or vendor-dependent. A scalable design with universality will lead to broader use and long-term sustainability of the proposed framework.

Methodology

Research Methodology and Development Stages

The implementation of the project carried out by structured stages to ensure clarity. All the stages are linked to certain tools and technologies that combine to achieve the formulated goals.

Stage 1: Requirement Analysis and System Design

Activities: identify security and resilience requirements, and define functional and non-functional specifications and design the overall Zero-Trust OTA architecture.

Tools/Technologies: Microsoft Office for system diagramming, IEEE literature for requirement validation.

Stage 2: Prototype Development

Activities: Test the individual units of the OTA system, and these include secure update distribution, rollback safe recovery engine, encryption and authentication mechanisms.

Tools/Technologies:

- **Hardware:** ESP32 and STM32 IoT boards.
- **Firmware Development:** Arduino IDE and PlatformIO.
- **Libraries:** OpenSSL, mbedTLS, mutual TLS (mTLS) and ECC (Elliptic-Curve Cryptography).

Stage 3: Implementation

Activities: Integrate components into a unified OTA framework with Zero-Trust enforcement, and configure secure communications between server and devices.

Tool/Technologies:

- **Server:** Python/Flask-based update server; Docker for containerization.
- **Networking:** Wireshark for packet inspection, HTTPS for delivery.
- **Version Control:** manifest based firmware versioning with hash validation.

Stage 4: Testing and Evaluation

Activities: Testing security, resilience and performance of the system.

Tools/Technologies:

- **Security Testing:** Kali Linux, OWASP ZAP, and custom test scripts.
- **Test reliability:** Simulating power and network failures.

Stage 5: Review and Documentation

Activities: Compare the system output with the already available OTA solutions, fine tune the design, and get ready to submit documentation.

Tools/Technologies: Microsoft Word to write the report, Excel and graphical tools to analyse the data, IEEE formatting of references.

Overall System

The secure OTA (Over-the-Air) firmware update framework under proposal is developed to be an end-to-end, resilient, and trustworthy system for IoT environments. It combines four dedicated elements, the Zero Trust Authorization Layer, the Lightweight Secure OTA Core, the Anomaly Detection Module, and the Resilient Fail-Safe Update Engine into an integrated architecture. The approach here guarantees that the entire lifecycle of the firmware update process, including initializing the request and post-installation validation, is governed by a strict set of security, reliability, and rollback safety policies. The overall system is organized into essential stages:

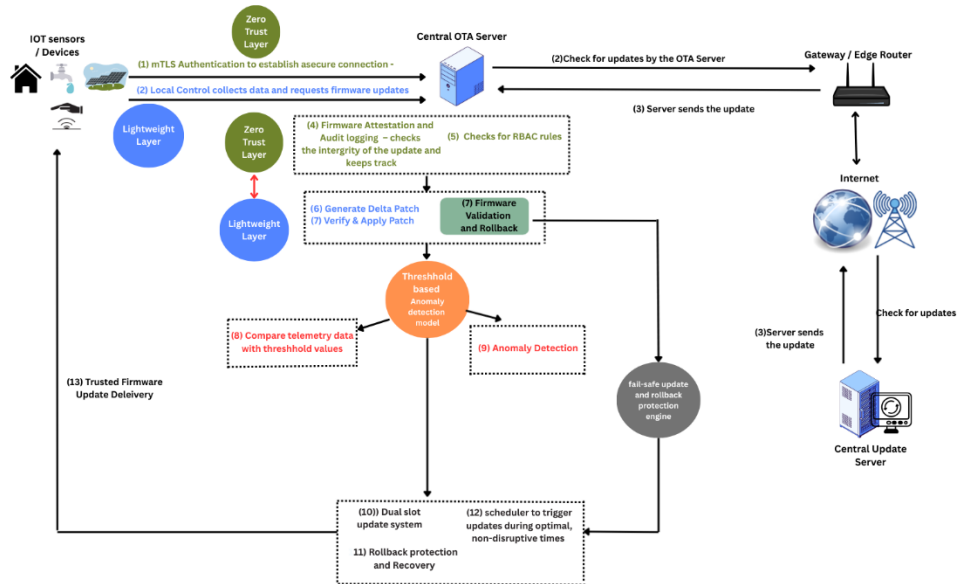


Figure 7.1 : Overall System

1. SAFETY Update Request & Authentication

The update process is starting when the IoT device requests for latest update from the cloud server or cloud server pushed an update to IoT devices.

The Zero Trust Authorization Layer enforces strong device-to-server trust by using mutual TLS (mTLS), firmware attestation and, certificate-based authentication.

In the initial stage Unauthorized devices or untrusted update sources are blocked, ensuring only verified entities participate in the OTA process.

2. Firmware Delivery with encryption

After being authenticated, the Lightweight Secure OTA Core downloads the firmware image. Updates are sent in encrypted form via resource-efficient cryptographic protocols (for integrity and confidentiality).

Partial downloads are securely cached in secure memory on the devices so that they can be continued in case transmission is disrupted. This ensures that despite the compromised bandwidth the update process is secure against manipulation or data leak.

3. Behavioral Anomaly Detection over Update

When the firmware package is being downloaded and prepared, the Anomaly Detection Module constantly checks the update behaviour.

Firmware size, update duration, and memory consumption parameters are benchmarked against pre-defined safe limits.

Suspicious deviations (e.g. oversized firmware or an unnaturally long update duration) lead to alerts and, in some cases, the update termination.

This proactive check helps to decrease the chances of stealth attacks, compromised payloads, or mistakenly configured updates entering to the system.

4. Secure Firmware Verification & Installation

Firmware is initially laid down in the Update Slot and checked against cryptographic signatures and version metadata.

This is only permitted when authenticity and version checks have been successful, and the firmware can then proceed to installation.

To detect incomplete or even malicious firmware installations, a two-step commit process is used so that the device goes on to the new firmware only after completion verification.

5. Rollback Protection & Recovery

If an updated firmware fails (e.g., crashes during boot, or fails during integrity checks), the Fail-Safe Engine reverts control to the previous stable firmware.

A Golden Recovery Image is permanently stored in a dedicated partition and provides a last resort self-healing utility.

This is to ensure that in a worst-case scenario, there is always at least one safe active state that the device can transition into, and that the device is never left in a state where it is unresponsive.

6. Logging & Forensic Analysis

Update events, failures, rollbacks, and anomalies are securely logged in non-volatile memory. Periodically, the recorded logs are uploaded into the cloud to be analyzed forensically, in order to help security auditing, debugging, and compliance reporting.

Individual System

The proposed resilient fail-safe update engine methodology is aimed to provide a safe, secure, and reliable Over-the-Air (OTA) update to IoT devices that operate in constrained environments. The strategy integrates cryptographic hashing, rollback, and fault recovery mechanisms to ensure the stability of devices even in the event of corruption or malicious updates. Methodology is subdivided into the following steps:

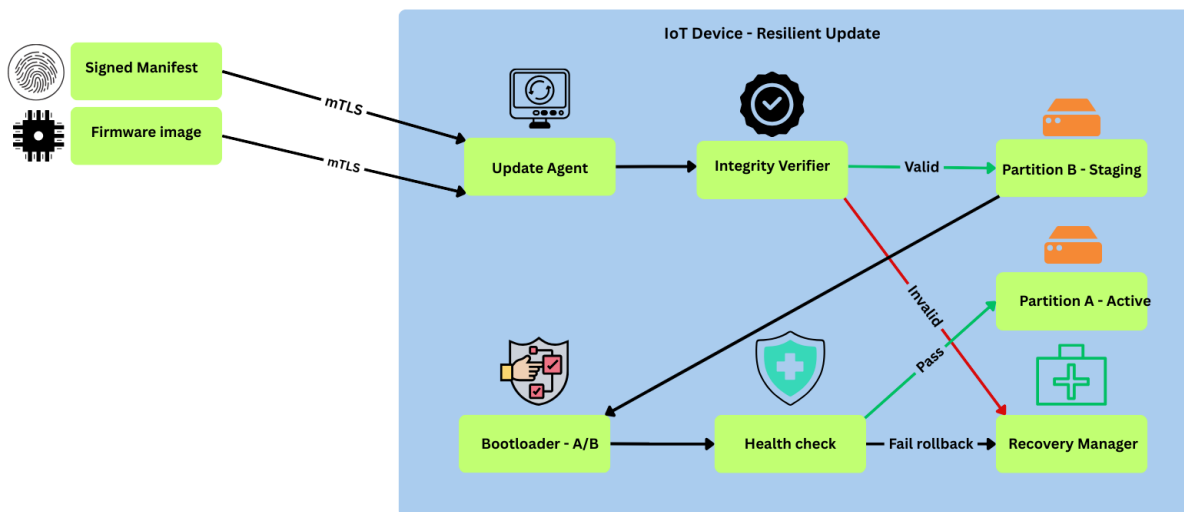


Figure 7. 2 : Individual System

1. System Design and Architecture

Define the architecture of the fail-safe update engine as a modular component within the overall OTA framework.

Allocate dedicated partitions in the device memory:

Active Firmware Slot - consists of the currently running verified firmware.

Update Slot - Temporary space into which an incoming update is stored.

Recovery Slot (Fallback): This slot holds a golden image (factory firmware) that will be used as a last-resort recovery.

Add metadata storage of version numbers, cryptographic hashes, and rollback counters.

2. Secure Firmware Acquisition

Be integrated with the OTA core to accept encrypted firmware images.

Compatibility checks are performed to ensure the authenticity of firmware before it is installed to prevent malicious payloads.

3. Version Control and Rollback Protection

Maintain a secure versioning system in well-secured memory.

- Compare the incoming firmware version with the stored counter. If the version is lower or the same, block the update (this mitigates downgrade attacks).
- In case the version is newer, carry out the installation.

Use monotonic counters or hash-chained version metadata to make the data tamper resistant.

4. Fail-Safe Update Mechanism

- Implement a **two-step update process**:
 1. Download and verify firmware in the **Update Slot**.
 2. Switch execution only after successful verification.
- Ensure **atomic commit**: if power loss or reset occurs during installation, the device reverts to the **previous stable firmware** instead of bricking.
- Add a **watchdog timer** that monitors the new firmware after boot.
 - If the firmware fails to boot or crashes repeatedly, trigger automatic rollback to the **last known good firmware**.

6. Recovery and Self-Healing Logic

- Keep a **golden recovery image** permanently in the Recovery Slot.
- In case both active and update firmware fail:
 - Automatically load the recovery image.
 - Restore device functionality and re-establish secure OTA connection.
- Log the failure event for forensic and debugging purposes.

6. Testing and Validation

- Carry out fault-injection testing to simulate update interruptions (e.g., loss of power, overwritten packets).

- Run test rollback scenarios using downgraded firmware, and check that rollback protection works properly.
- Ensure the processed amount of memory and the process time are within tolerable limits on embedded devices.

7. Tools and platforms

- **Hardware:** ESP32 or STM32 boards.
- **Libraries:** TinyCrypt, MbedTLS.
- **OTA Frameworks:** ESP-IDF OTA APIs / STM32Cube OTA component (modified to support fail-safe logic).
- **Simulation:** QEMU is an embedded firmware fault injection.
- **Version Tracking:** Non-Volatile storage (NVS) to implement secured monotonic counters.

Timeline

Zero-Trust OTA Component Project Timeline

	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr
Planning											
Research											
Data Collection											
Development											
Testing											
Integration & Testing											
Reporting											

Figure 8 . 1 : Timeline Gantt chart

Project Requirements

Resilient Fail-Safe Update Engine is designed to ensure that IoT devices can maintain functionality and security in the event of a failed firmware update, pausing or ongoing malicious tampering of the firmware or its content. This component is focused on the rollback safety, recovery logic, and safe firmware version control to avoid device bricking and provide always-on availability. To achieve this the system needs the following:

1. Hardware Requirements

- IoT development board(e.g. ESP32 or STM32).
- At least three firmware storage partitions:
- Active Slot (running firmware).
- Update Slot (staging area for new firmware).
- Recovery Slot (golden firmware image).
- Non-Volatile Memory (NVM) for version tracking and metadata storage.

2. Software Requirements

- Partition management system to handle multiple firmware images.
- Watchdog timer for runtime monitoring of new firmware after boot.
- Rollback logic implemented at bootloader/firmware level.
- Error-logging mechanism to store update failure records.

3. Security Requirements

- Secure versioning system to prevent downgrade attacks.
- Protection of rollback metadata against tampering.
- Safe-switching mechanism to ensure atomic commit during firmware swaps.

Functional Requirements

The functional requirements describe what the Fail-Safe Update Engine must perform to meet its intended objectives.

1. Firmware Slot Management

- The flash memory of the system should be partitioned into three logical partitions, **active**, **update**, and **recovery**, each having a minimum size (say 512 KB/partition).
- When the engine switches between slots it should **not corrupt** any **application data** stored.

2. Rollback Protection

- Any attempt to install a firmware image that is **identical or older** than the one in use must be blocked by the engine. This verification is performed by a cryptographic

counter (such as a monotonic counter in secure flash). That counter only increments on passing a SHA-256 integrity check.

3. Atomic update handling

- The updates must be transactional: in case the **power source** is **lost**, or the device reboots in the process of an upgrade, subsequent booting should be followed by a reboot to **the last reliable** firmware. There is also the fact that only upon success of the digital signature verification the engine writes to the update slot.

4. Watchdog-Based Monitoring

- The first boot of a new firmware is monitored by a watchdog timer (configurable between 5-30 seconds). Provided the machine crashes, resets, or enters a boot-loop three consecutive times, the rollback logic automatically activates and automatically returns to the last stable firmware.

5. Golden Recovery Image

- The system stores a factory image in a **write-protected recovery** slot. In the case of either the active or update slot failure, the recovery image is loaded instead.

6. Failure Logging

- Rollback events, integrity check failures and watchdog resets are written to a persistent log. Such logs can be exported over UART or secure channel to be examined by the forensic crafter.

Non-Functional Requirements

Describe the quality attributes and performance expectations of the update engine.

1. Reliability

- The system must guarantee device bootability under all circumstances (no bricking).
- Rollback and recovery must always succeed within seconds of failure detection.

2. Performance

- Update operations should not exceed the memory or processing limits of low-power IoT devices.

- Switching between firmware slots must introduce minimal downtime.

3. Scalability

- The mechanism must run on both **8-bit microcontrollers (e.g. STM32)** and higher-end boards (e.g., ESP32) without architecture specific significant **code modifications**.

4. Maintainability

- The update engine codebase must be **modular**, with rollback logic integrated in a separate module.
- Metadata handling (e.g., counters, hashes) must allow extension for **future cryptographic algorithms** with minimal changes.

5. Security

- All version counters and metadata must be stored in **tamper-proof flash memory regions** (using hardware MPU or flash lock bits).
- The recovery slot must remain **read-only at runtime**, ensuring firmware integrity even under attempted overwrite attacks.

6. Usability

- Recovery from a failed update must **not require user interaction**.
- Debug logs must use **standardized event codes** (e.g., ERR-01 for rollback, ERR-02 for signature fail) for ease of developer interpretation.

Testing and Evaluation

A systematic testing approach will be used to support reliability, security, and resilience of the proposed rollback-safe OTA firmware update system. The testing will check on functional correctness of the system and ensure security will be held under various operational and failure circumstances.

1. Testing Objectives

Ensure that firmware updates delivers in a secure way by authorized and trusted sources.

Ensure rollback-safe mechanisms are reliable in case the update is interrupted.

2. Types of Testing

- **Unit Testing**

Isolated modules including secure communication channel, version verification and recovery engine will be tested isolation.

verify that version numbers are not accepting previous firmware packages.

- **Integration Testing**

The interaction of modules (delivery of firmware + secure installation + rollback logic) will be tested as a whole process.

This provides end-to-end consistency of update process.

- **Security Testing**

Simulate adversarial conditions:

- Firmware freshness (altered signature/hash).
- Replay attacks (retransmission of old updates).

Make sure that the system prevents malicious activity by rejecting and malicious attempts and log incidents.

- **Resilience Testing (Failure Handling)**

Simulate power failure or network disturbance during update.

The device has to automatically revert back to the previous stable version.

- **Performance Testing**

Count time spent to complete updates.

Analyse parameters of memory consumption at the process of update and rollback.

Compare the speed and reliability of update speed with a baseline (as an example, using a traditional OTA without a rollback).

3. Test Scenarios

Normal Update Scenario - Valid firmware is notified to device, update completes successfully.

Interrupted Update - It is possible that the device loses power in the middle of the installation process; recovery engine goes back one step and retrieves the last stable version.

Altered Firmware - Altered package fails signature check and is rejected.

Rollback Attempt - Attacker tries to downgrade with obsolete firmware; the system blocks it.

Resource-Constrained Update Test - The test has been conducted on a limited flash memory and is therefore very efficient.

4. Evaluation Metrics

Success Rate of successful completion of firmware update.

Mean Recovery Time it takes to get to back on working status after a failed update.

Detection Accuracy of fault/failure situations.

Resource Overhead (CPU, memory and storage consumption).

Security Effectiveness (percentage of successful attack scenarios mitigated).

5. Tools and Incremental Environment

Embedded systems such as ESP32, STM32 or Raspberry Pi).

Cryptographic libraries (mTLS).

Embedded OS(FreeRTOS or lightweight Linux).

Simulation software for power / network interruption.

6. Expected Outcome

The system shall exhibit:

- Trustable delivery of updates with cryptographic validation of integrity.
- Have automatic recovery of failed update states without user involvement.
- Practical prevention of firmware rollback and tampering attacks.
- The low performance overhead which fits well in resource-constrained IoT devices.

Commercialization and Entrepreneurship Potential

The Resilient and trust-based OTA framework targets to fill a gap in the IoT environment, ensuring the need to have reliable firmware updating without bricking of the device and long outage periods. Most of the current solutions focus more on high cryptographic security without much consideration of the **resilience** of IoT devices within the constraint environment, which exposes businesses to the risk of expensive failures during update periods.

Practical Industry Adoption

- **Smart Metering & Energy Grids:** With confidence that device downtime does not create problems with billing disruptions and regulatory compliance, utilities can deploy millions of smart meters. This Resilient and trust-based OTA framework ensures that things remain smooth, thus you do not need to spend money on technicians to correct things.
- **Industrial IoT (IIoT):** SCADA systems and manufacturing plants should be in twenty-four-hour operation. Our Resilient and trust-based OTA framework is highly effective since it is lightweight and resilient.
- **Smart Homes & Consumer Devices:** Low cost smart plugs, cameras and devices can be plugged into our library and inform customers that the updates will never damage the devices.

Market Need

- **Global IoT Expansion:** IoT expansion is projected to hit more than 25 billion IoT devices deployed globally in 2030 through the spread of smart homes, industrial IoT, healthcare, energy, and critical infrastructure.
- **Expensive consequences of device bricking:** Failed devices lead to device downtime and customer and maintenance expenses. For enterprises that deploy thousands of IoT sensors or controllers, the cost of physical recall or manual repair is extremely high.
- **Regulatory Push to Security:** The EU and US and many regions are introducing and enforcing **cybersecurity and resilience standards** for IoT devices. Vendors should be able to prove reliability of updating and rollback in order to be in compliance.

Value Proposition

- **Reliability First:** The system is designed to ensure that even in circumstances where a failure occurs, it will be safe to rollback and recover without device bricking.

- **Business Continuity:** Businesses will be able to implement large-scale IoT systems with minimal chance of experiencing disruption because of failed updates.
- **Lightweight and Flexibility:** The engine can be distributed as a lightweight firmware library/module or middleware that can be easily integrated into existing IoT platforms.

Business Potential

Target Market: Manufacturers of IoT devices, makers of embedded systems and IoT network deployers at large scale.

- **Revenue Models:**
 - **Licensing** of the rollback-safe OTA engine as a software component to manufacturers of IoT.
 - **Consultancy & Integration Services** to enterprises with the need to have safe and resilient update systems.
 - **Collaborations** with IoT platform providers (AWS IoT, Azure IoT Hub) to develop lightweight resilience modules.
- **Cost Benefit:** It offers resilience without demanding high-cost of hardware, which making it more accessible to small-scale manufacturers.

Go-to-Market Strategy

- **Phase 1 - Pilot Integrations** - Partner with local IoT start-ups in energy and health to demonstrate field operations.
- **Phase 2 - licensing model** - provide SDK licenses to OEMs using optional support contracts.
- **Phase 3 - Cloud Partnerships** - integrating with AWS IoT and Azure IoT Hub as a lightweight resilience layer.
- **Phase 4 - Spin-off Startup** Build a dedicated venture that provides managed OTA resilience services for enterprise IoT systems.

Entrepreneurship Potential

The solution is obvious in that it has merit when it comes to commercial spin-off of the new venture in the form of a startup enterprise.

- **Firmware update resilience SDK/ library** for IoT developers.
- **Plug and play Firmware recovery modules** to manufacturers.
- **Managed services** to enterprises that want to deploy and secure large scale of IoT devices.

Social and User Advantages

- **Increased trust** in IoT devices by guaranteeing that updates do not provide any side effects to device stability.
- Reduced **e-waste**, since there is less equipment that needs to be substituted because of failed updates.
- Reduce the **costs of maintenance** to the businesses and government organisations that perform large-scale deployments of the IoT.
- Contribution to **cybersecurity compliance frameworks**, supporting to help vendors comply with international standards.
- **Environment Benefit** due to reduced e-waste since devices can be used for longer time.

Budget and Budget Justifications

Item	Quantity	Unit Cost (LKR)	Total Cost (LKR)	Justification
Laptop/PC (Already existing)	-	-	0	Already available personal resources.
IoT Development Boards (e.g., ESP32 / STM32)	4	6,500	26,000	ESP32 boards are cost-effective and widely used in IoT security research.
Secure Boot/Encryption Modules (e.g., ATECC608A Crypto chip)	2	3,500	7,000	Used to simulate hardware-based cryptographic security and support ECC features.
Storage Devices (USB drives / SD cards)	4	1,000	4,000	For transfer firmware, test data storage, and recovery testing.
Networking Devices (Wi-Fi Router / Switch)	1	8,500	8,500	Required to simulate enterprise-like IoT device connectivity for OTA delivery.
Software Tools & Libraries	-	-	0	Most of the development tools (Arduino IDE, PlatformIO, OpenSSL, Wireshark, etc.) are open-source.
Cloud Resources	-	-	0	Cloud services such as AWS

(Testing – Free Tier)				IoT Core / Azure IoT Hub will be used within free student credits or free tier.
Printing & Documentation	-	-	3,000	Printing of project proposal, final report, and Other essential materials.
Cables, Connectors, Power supplies	-	-	5,000	To power-up and connecting development boards, routers, and test setups.

Table 11 . 1 : Budget and Budget Justification

Total Estimated Cost: LKR 53,500

Budget Justification

The budget proposed is carefully planned to ensure the implementation (cost will not be too high compared to the Sri Lankan academic environment).

- **Hardware Boards:** aside from the affordability of the ESP32/STM32 boards, they are powerful enough to simulate actual IoT devices so that OTA updates can be tested. Each of the members will need its own independent development and test boards.
- **Crypto Modules:** ATECC608A secure elements have been included to represent a small set of the crypto modules; this allows hardware-based cryptography to be realistically illustrated without excess units.
- **Networking:** A common Wi-Fi router would provide a shared testbed on which all OTA update experiments would be executed in conditions that reflect practical IoT deployment situations.
- **Software Tools:** Development, and cryptographic libraries are free/open-source and thus, costs are reduced to a minimum.
- **Cloud Services:** Student plans and free options available with compute services will be used to simulate an OTA delivery environment free of subscription cost.

- **Printing & Miscellaneous:** Required in the creation of printable reports or documentation as well as smaller project installation requirements.

Overall, this budget enables a trade-off between economic and technical feasibility, low cost, but adds an academic exposure, hence, the feasibility of the project being successfully implemented.

References

[1] IBM, “What is the Internet of Things (IoT)?,” IBM, Jul. 29, 2025. [Online]. Available: <https://www.ibm.com/think/topics/internet-of-things>. [Accessed: Jul. 29, 2025].

[2] Fortinet, “What is the Internet of Things (IoT)? Benefits, Challenges & Applications,” Fortinet. [Online]. Available: <https://www.fortinet.com/resources/cyberglossary/iot>. [Accessed: Jul. 29, 2025].

[3] TechTarget, “What is OTA update (over-the-air update)? Definition,” SearchMobileComputing, Dec. 9, 2022. [Online]. Available: <https://www.techtarget.com/searchmobilecomputing/definition/OTA-update-over-the-air-update>. [Accessed: Jul. 30, 2025].

[4] Kaspersky, “What is IoT? Tips for IoT Security,” Kaspersky, Jul. 30, 2025. [Online]. Available: <https://www.kaspersky.com/resource-center/definitions/what-is-iot>. [Accessed: Jul. 30, 2025].

[5] Analog Devices, “Over-the-Air (OTA) Updates in Embedded Microcontroller Applications: Design Trade-Offs and Lessons Learned,” Analog Devices. [Online]. Available: <https://www.analog.com/en/resources/analog-dialogue/articles/over-the-air-ota-updates-in-embedded-microcontroller-applications.html>. [Accessed: Jul. 30, 2025].

[6] Electronics World, “OTA update software design for embedded applications,” Electronics World, Sep. 9, 2020. [Online]. Available: <https://www.electronicsworld.co.uk/ota-update-software-design-for-embedded-applications/28323/>. [Accessed: Jul. 30, 2025].

[7] StackExchange, “What is the purpose of anti-rollback protection in IoT devices?,” Information Security Stack Exchange. [Online]. Available: <https://security.stackexchange.com/questions/233651/what-is-the-purpose-of-anti-rollback-protection-in-iot-devices>. [Accessed: Jul. 30, 2025].

- [8] Stormotion, “Updating IoT Devices in 2025: Best Practices for Secure and Efficient IoT Firmware Updates,” Stormotion.io. [Online]. Available: <https://stormotion.io/blog/updating-iot-devices/>. [Accessed: Jul. 30, 2025].
- [9] Datamation, “The Big List of IoT Devices and IoT Products,” Datamation, Nov. 10, 2023. [Online]. Available: <https://www.datamation.com/mobile/85-top-iot-devices/>. [Accessed: Jul. 30, 2025].
- [10] Mender.io, “Over-the-air (OTA) update best practices for industrial IoT and embedded devices,” Mender.io. [Online]. Available: <https://mender.io/resources/reports-and-guides/ota-updates-best-practices>. [Accessed: Jul. 31, 2025].
- [11] Emteria, “Over-The-Air software updates explained easily,” Emteria, Jan. 19, 2024. [Online]. Available: <https://emteria.com/blog/over-the-air-software-updates>. [Accessed: Jul. 31, 2025].
- [12] “Secure firmware Over-The-Air updates for IoT: Survey, challenges, and discussions,” *J. Parallel Distrib. Comput.*, vol. xx, May 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S2542660522000142>. [Accessed: Aug. 1, 2025].
- [13] GovLoop, “The Problem With Traditional IoT,” GovLoop, Mar. 28, 2018. [Online]. Available: <https://www.govloop.com/problem-traditional-iot/>. [Accessed: Aug. 1, 2025].
- [14] Springer, “A robust and efficient blockchain-based framework for updating firmware in IoT environments,” *Peer-to-Peer Networking and Applications*, Springer, Jun. 9, 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s12083-025-02031-7>. [Accessed: Aug. 1, 2025].
- [15] “The Lazarus Effect: Healing Compromised Devices in the Internet of Small Things,” *arXiv preprint*, arXiv:2005.09714, May 19, 2020. [Online]. Available: <https://arxiv.org/abs/2005.09714>. [Accessed: Aug. 1, 2025].
- [16] FBI Support, “What are the best practices for securing firmware and software updates for IoT devices?,” FBI Support Cyber Law Knowledge Base. [Online]. Available: <https://fbisupport.com/best-practices-securing-firmware-software-updates-iot-devices/>. [Accessed: Aug. 2, 2025].
- [17] Cornell University, “Incremental Firmware Update Over-the-Air for Low-Power IoT Devices over LoRaWAN,” *arXiv preprint*, arXiv:2505.13764, May 19, 2025. [Online]. Available: <https://arxiv.org/abs/2505.13764>. [Accessed: Aug. 3, 2025].

[18] “Secure firmware Over-The-Air updates for IoT: Survey, challenges, and discussions,” *J. Parallel Distrib. Comput.*, vol. xx, May 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S2542660522000142>. [Accessed: Aug. 3, 2025].

[19] Springer, “A robust and efficient blockchain-based framework for updating firmware in IoT environments,” *Peer-to-Peer Networking and Applications*, Springer, Jun. 9, 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s12083-025-02031-7>. [Accessed: Aug. 3, 2025].

[20] Tuya Smart, “IoT Core | Tuya Smart,” Tuya Smart. [Online]. Available: <https://www.tuya.com/platform/clouddev/iot-core>. [Accessed: Aug. 6, 2025].

[21] Cloudflare, “What is IoT security? IoT device security,” Cloudflare. [Online]. Available: <https://www.cloudflare.com/learning/security/glossary/iot-security/>. [Accessed: Aug. 4, 2025].